

MSBA 265: Business Analytics Topics

Foundational Module 1: Practical Homework Report

Interactive EDA, Data Quality Verification, Data Dictionaries, & Outlier Pipelines

Ziyan Chen

Instructor: Shyla Solis

Report Overview

This report documents the Module 1 workflow on the French Motor Third-Party Liability Claims dataset (freMTPL2freq, OpenML ID 41214): git-tracked workspace setup, programmatic data procurement, raw boundary audits, a Business Data Dictionary, distribution and correlation plots with written interpretations, and a production Tukey $1.5 \times IQR$ outlier pipeline. All code, figures and this report are hosted in the public GitHub repository [msba265-module1-ziyanchen](#).

1 Part 1: Workspace Setup, Git Ingestion & Procurement

I set up the project in VS Code with a virtual environment (venv) and initialized Git. The .gitignore excludes venv/, __pycache__/ and .ipynb_checkpoints/, so only source files and deliverables are tracked.

The raw dataset was downloaded programmatically with data/download_data.py, which fetches freMTPL2freq.csv from OpenML and saves it as data/raw_business_data.csv. The download returned 678,013 rows $\times$ 12 columns, matching the expected record count.

All scripts, the notebook, figures, the data dictionary, the cleaned dataset and this report are committed to Git and pushed to the public GitHub repository.

1.1 Repository structure

```
msba265-module1-ziyanchen/
|-- .gitignore
|-- README.md
|-- requirements.txt
|-- Module1_Homework_Report.pdf
|-- data/
|   |-- download_data.py
|   |-- raw_business_data.csv
|   |-- cleaned_business_data.csv
|-- notebooks/
|   |-- 01_eda_and_data_dictionary.ipynb
|-- src/
|   |-- clean_outliers.py
|-- reports/
|   |-- data_dictionary.csv
|   |-- figures/
|       |-- feature_distributions.png
|       |-- correlation_heatmap.png
```

2 Part 2: Raw Boundary Audits & Business Dictionary Artifact

2.1 Structural audit and raw boundary verification

`df.info()` shows 678,013 records, 12 columns and zero nulls in every feature. `df.describe().T` was used to check the raw minimum and maximum of the three key fields:

- **DrivAge:** Min = 18, Max = 100. No negative ages, no child policyholders, no sentinel codes such as -999.
- **Exposure:** Min = 0.0027 (≈ 1 day), Max = 2.01. No negative or zero durations; a few policies run longer than one year.
- **BonusMalus:** Min = 50, Max = 230. All records sit inside the French regulatory range of 50 – 350.

2.2 Business Data Dictionary

Cell 4 builds a 12-row Business Data Dictionary (feature name, business label, data type, unit, null count, unique values, business definition) and exports it to `reports/data_dictionary.csv`. The file is included in the repository.

2.3 Defense: Exposure as a rate offset, and Density vs. Area collinearity

Why Exposure acts as a rate offset

Exposure is the fraction of a year each policy was active, so a policy with Exposure = 1.0 had twice as much time to generate claims as one with 0.5. This means raw ClaimNb is not comparable across policies; what we actually want to model is the claim rate, ClaimNb / Exposure. In a Poisson regression this is done by adding $\log(\text{Exposure})$ to the model with its coefficient fixed at 1 — this is called an offset. It adjusts for time-at-risk without being estimated as a regular predictor.

Why continuous Density and categorical Area cannot both be included in a linear regression model

Density and Area cannot both go into a linear regression because they measure the same thing: Area is just Density cut into 6 bins (A = rural, F = urban). Including both makes the two features almost perfectly collinear, so the model cannot tell which one is driving the effect, the coefficients become unstable, and in the extreme case the regression has no unique solution. We keep only one of them — $\log(1 + \text{Density})$ or Area, not both.

3 Part 3: Overriding Automated Rules & Correlation Visuals

3.1 Pearson correlation heatmap

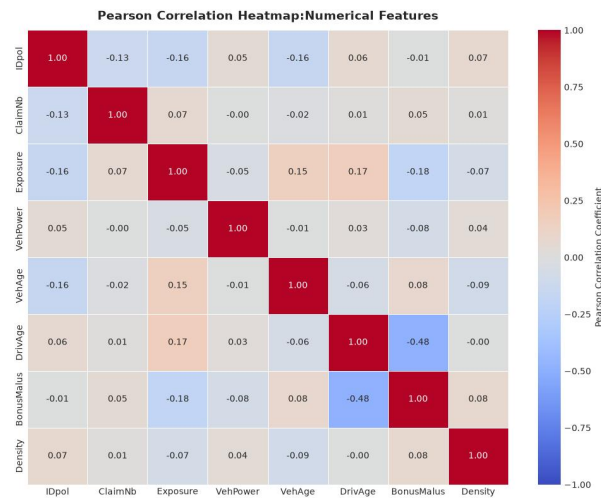


Figure 1: Pearson correlation heatmap of numerical features (reports/figures/correlation_heatmap.png)

Required report statements: correlation heatmap

- **Distribution Shape:** Almost every cell is pale, meaning the numerical features are nearly uncorrelated with each other. All values fall between -0.18 and $+0.17$, except one clearly blue cell: DrivAge vs. BonusMalus at $r = -0.48$. No pair reaches the $|r| \geq 0.85$ severe-multicollinearity threshold.
- **Key Metrics:** DrivAge–BonusMalus ($r = -0.48$) is a moderate negative relationship — older drivers accumulate claim-free years and earn discounts toward 50, while younger drivers stay near or above 100. ClaimNb is nearly uncorrelated with every feature ($|r| \leq 0.07$), which is typical for a rare-event target.
- **Recommended Analytics Action:** No numerical feature needs to be dropped for collinearity; DrivAge and BonusMalus can both stay. Two cautions: Pearson only detects linear patterns, so a U-shaped age–claim curve would still show $r \approx 0$; and Area is not in this matrix but duplicates Density, so only one of them should enter the model.

3.2 Executive defense: why we do NOT log-transform ClaimNb

Domain Override: ClaimNb stays in raw count units

The automated rule flagged ClaimNb (skew = 5.60) and recommended a log transform. We override this for two reasons. First, it is mathematically impossible: over 95% of policies have 0 claims, and $\log(0) = -\infty$, so the transform cannot be computed. Second, ClaimNb is a discrete count of accidents, not a continuous volume metric like Density, so its heavy skew is normal, not a data problem. We keep ClaimNb in raw count units and model it with Poisson regression, using $\log(\text{Exposure})$ as an offset.

4 Part 4: Distribution Audits & Written Plot Statements

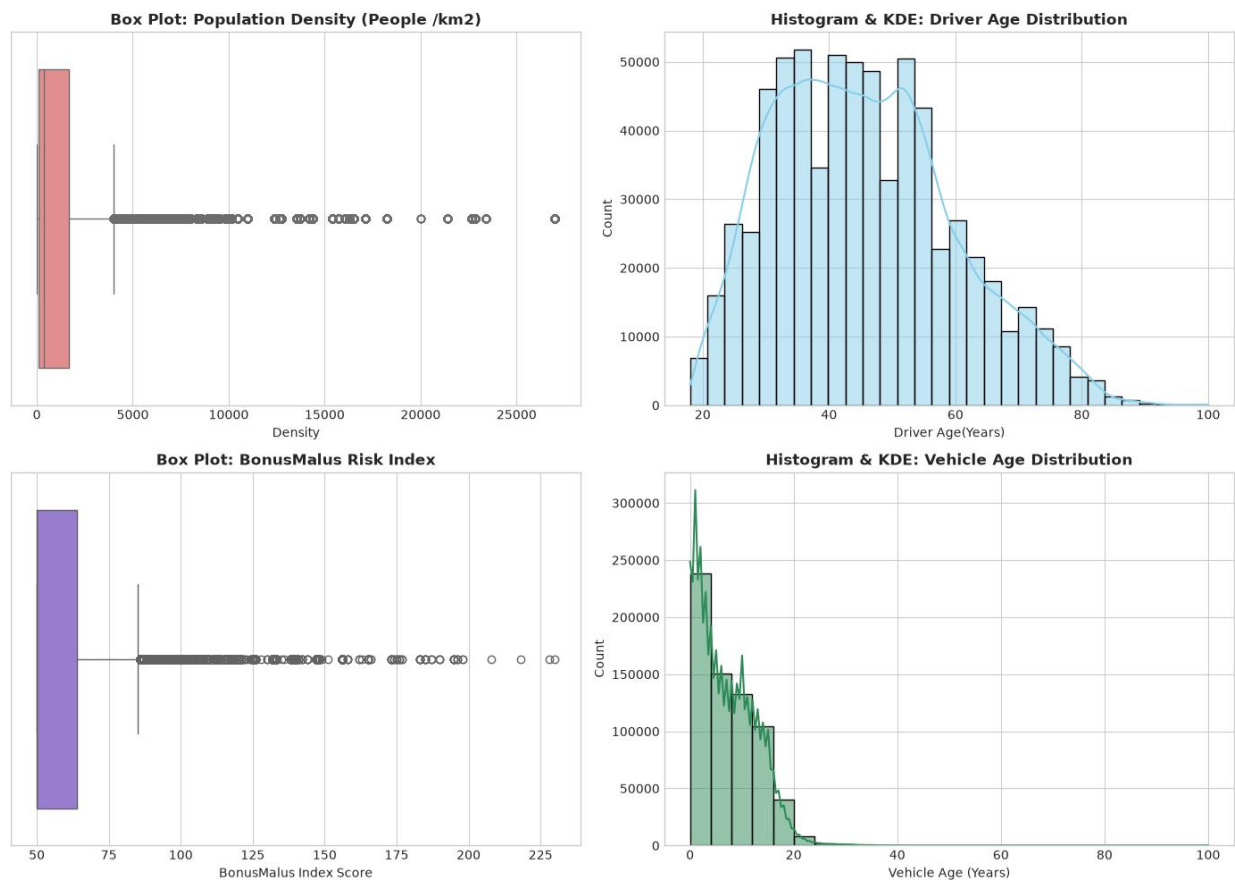


Figure 2: Box plots and KDE histograms for Density, DrivAge, BonusMalus and VehAge
(reports/figures/feature_distributions.png)

Chart 1 — Density Box Plot

- **Distribution Shape:** Extremely right-skewed (skew = 4.65). The box is squeezed against zero and a long trail of outlier dots extends all the way to 27,000 people/km².
- **Key Metrics/Whiskers:** Q1 = 92, median = 393, Q3 = 1,658. The upper Tukey fence is 4,007 and the whisker stops at 3,997; everything beyond that (77,566 records, 11.4%) is drawn as outlier dots.
- **Recommended Analytics Action:** These dots are real policyholders in large cities such as Paris, not data errors, so we do not drop them. Apply $\log(1 + \text{Density})$ before linear modeling to compress the right tail.

Chart 2 — DrivAge Histogram & KDE

- **Distribution Shape:** Roughly symmetric, bell-shaped (skew = 0.44), with a slight right tail toward the oldest drivers. The KDE drops smoothly to zero near 100.
- **Key Metrics/Whiskers:** Q1 = 34, median = 44, Q3 = 55. Minimum 18 and maximum 100 — all valid legal driving ages, no negative or impossible values.

- **Recommended Analytics Action:** No log transform or outlier trimming needed. Standard Z-score scaling is sufficient.

Chart 3 — BonusMalus Box Plot

- **Distribution Shape:** Concentrated at the low end with a long right tail (skew = 1.73). Most policyholders sit at or near the maximum discount of 50.
- **Key Metrics/Whiskers:** Q1 = 50, median = 50, Q3 = 64. The upper Tukey fence is 85, so scores above 85 appear as individual dots extending up to the maximum of 230.
- **Recommended Analytics Action:** The dots are legitimate regulatory surcharges, not errors, so keep them. BonusMalus is an index, not a physical quantity, so do not log-transform it; scale it and add a binary flag `Is_Surcharge = BonusMalus > 100`.

Chart 4 — VehAge Histogram & KDE

- **Distribution Shape:** Moderately right-skewed (skew = 1.15), peaking at 0–5 years and falling off steadily after 20 years.
- **Key Metrics/Whiskers:** Q1 = 2, median = 6, Q3 = 11. The upper Tukey fence is 24.5, but the maximum reaches 100, which is not a realistic age for an insured car.
- **Recommended Analytics Action:** Cap VehAge at a realistic limit (around 30 years) or investigate the 100-year records as data entry placeholders, then bin into brackets (0–5, 6–10, 11+ years) or apply min-max scaling.

5 Part 5: Production Pipeline Scripting & Final PDF Export

The Tukey $1.5 \times \text{IQR}$ filter tested in the notebook was turned into a standalone script, `src/clean_outliers.py`. Running `python src/clean_outliers.py` from the project root reads `data/raw_business_data.csv`, removes Density records outside the Tukey fences and writes `data/cleaned_business_data.csv`. Both the script and the cleaned CSV are in the repository.

5.1 Notebook outlier audit (Cell 8)

```
=== OUTLIER AUDIT REPORT FOR 'Density' ===
Original Record Count: 678,013 records
Tukey IQR Range [-2257.00, 4007.00] : Excludes 77,566 records
Z-Score Threshold (|z| <= 3.0):      Excludes 15,209 records
```

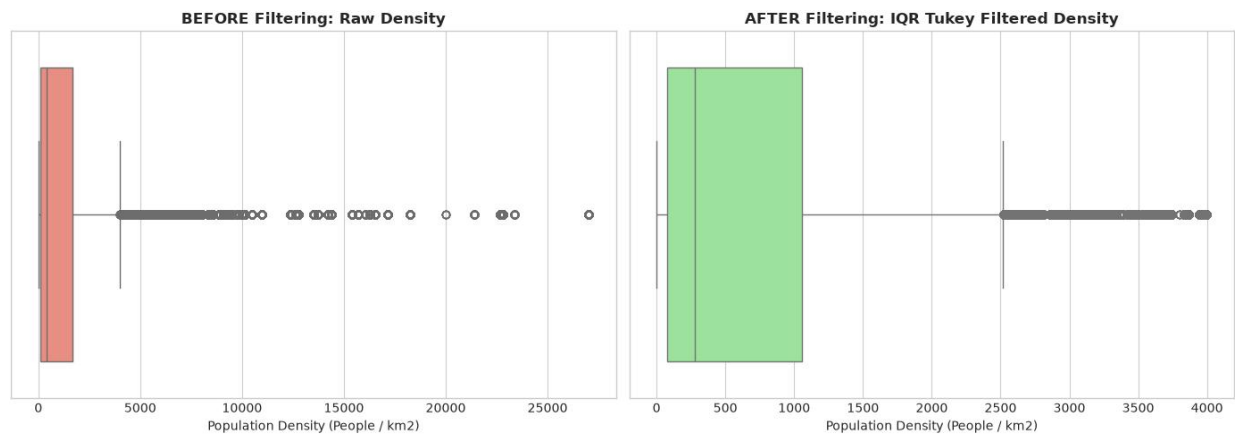


Figure 3: Density before and after Tukey IQR filtering (notebook Cell 8)

The AFTER plot shows the effect of the filter: once values above 4,007 people/km² are removed, the box (92 – 1,658) and whiskers become clearly visible instead of being squeezed against zero. Because Density is heavily right-skewed, the Tukey method (based on Q1 and Q3) gives a more reliable boundary than the Z-score method, whose mean and standard deviation are inflated by big cities such as Paris.

5.2 Production script terminal output

```
(venv) PS C:\...\module1> python src/clean_outliers.py
=====
      PRODUCTION OUTLIER FILTERING REPORT
=====
Target Feature Filtered: Density
Initial Dataset Records: 678,013
Tukey IQR Valid Range:  [-2257.00, 4007.00]
Outlier Records Removed: 77,566 (11.44%)
Final Cleaned Records:  600,447

[+] Saved cleaned dataset artifact to: data\cleaned_business_data.csv
```